UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

GII 24.36 Herramienta de gestión para el seguimiento de pacientes con vídeo a través de dispositivo móvil.

Presentado por Andrés Puentes González
en Universidad de Burgos — 13 de febrero de 2026
Tutores: José Luis Garrido Labrador
José Miguel Ramírez Sanz

UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

D. José Luis Garrido Labrador, junto a D. José Miguel Ramírez Sanz profesores del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Exponen:

Que el alumno D. Andrés Puentes González, con DNI 13172510L, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado GII 24.36 Herramienta de gestión para el seguimiento de pacientes con vídeo a través de dispositivo móvil.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 13 de febrero de 2026

Vº. Bº. del Tutor:

D. José Luis Garrido Labrador

Vº. Bº. del co-tutor:

D. José Miguel Ramírez Sanz

Resumen

El presente Trabajo de Fin de Grado, titulado *«Herramienta de gestión para el seguimiento de pacientes con vídeo a través de dispositivo móvil»* (proyecto GII 24.36), se desarrolla en el marco de una colaboración con el departamento de neurología del Hospital Universitario de Burgos. El objetivo principal es el desarrollo de una aplicación móvil nativa para Android, diseñada para facilitar la telerrehabilitación asíncrona entre terapeutas y pacientes. La solución propuesta implementa una arquitectura cliente-servidor que permite a los profesionales sanitarios diseñar rutinas de ejercicios personalizadas y evaluar el desempeño de los pacientes de forma remota.

Un aspecto crítico del proyecto ha sido el diseño de la experiencia de usuario, adaptando la interfaz a las limitaciones motoras de los pacientes mediante el uso de comandos simplificados y flujos guiados. Asimismo, se ha implementado un módulo de captura de vídeo, resolviendo desafíos técnicos complejos relacionados con el ciclo de vida de la cámara y la gestión de archivos locales. Esta captura de vídeo de alta calidad es fundamental, ya que constituirá la base de datos necesaria para futuros módulos de Inteligencia Artificial destinados a la detección de poses y la corrección automática de ejercicios.

El resultado final es una herramienta funcional que permite gestionar roles de usuario (administrador, terapeuta, paciente), realizar y grabar sesiones de rehabilitación, y visualizar la evolución del paciente mediante gráficas de rendimiento, sentando las bases tecnológicas para un sistema de telerrehabilitación inteligente y autónomo.

Descriptores

Enfermedad de Parkinson, telerrehabilitación, bradicinesia, API REST, Android, servidor, administrador, terapeuta, paciente.

Abstract

This Bachelor's Thesis, titled *"Management tool for patient follow-up with video via mobile device"* (project GII 24.36), is developed within the framework of a collaboration with the Neurology Department of the University Hospital of Burgos. The main objective is the development of a native Android mobile application designed to facilitate asynchronous telerehabilitation between therapists and patients. The proposed solution implements a client-server architecture that allows healthcare professionals to design personalized exercise routines and assess patient performance remotely.

A critical aspect of the project has been the User Experience (UX/UI) design, adapting the interface to the motor limitations of patients through the use of simplified commands and guided flows. Furthermore, a video capture module has been implemented, solving complex technical challenges related to the camera lifecycle and local file management. This high-quality video capture is fundamental, as it will constitute the necessary database for future Artificial Intelligence modules aimed at pose detection and automatic exercise correction.

The final result is a functional tool that allows for the management of user roles (administrator, therapist, patient), the performance and recording of rehabilitation sessions, and the visualization of patient progress through performance charts, laying the technological foundations for an intelligent and autonomous telerehabilitation system.

Keywords

Parkinson's disease, telerehabilitation, bradykinesia, REST API, Android, server, administrator, therapist, patient.

Índice general

Resumen **i**

Abstract **ii**

Índice general **iii**

1. Introducción **1**
1.1. Contexto académico 2
1.2. Justificación y motivación del proyecto 2
1.3. Objetivos generales 3
1.4. Entregables y repositorio 3

2. Objetivos del Proyecto **5**
2.1. Objetivos funcionales 5
2.2. Objetivos técnicos 6
2.3. Objetivos personales 7

3. Conceptos Teóricos **9**
3.1. Contexto sanitario 9
3.2. Ingeniería del software y metodología 10
3.3. Patrones de diseño y arquitectura aplicada 11
3.4. Conceptos Técnicos de Implementación 12
3.5. Seguridad y privacidad 13

4. Técnicas y Herramientas **15**
4.1. Herramientas de programación 15
4.2. Bibliotecas 16
4.3. Gestión de datos 17

4.4. Gestión de proyecto 18

5. Aspectos Relevantes del Desarrollo **19**
5.1. Fase de selección de software 19
5.2. Fase de definición de requisitos y funcionalidades 21
5.3. Fase de diseño e implementación 23
5.4. Fase de desarrollo de la API REST 24
5.5. Fase de creación de aplicación móvil 26
5.6. Fase de gestión de vídeos 28

6. Trabajos Relacionados **31**
6.1. Antecedentes Académicos: TFM 31
6.2. Antecedentes Académicos: TFG 33
6.3. Soluciones Comerciales de Referencia 35

7. Conclusiones y Líneas de trabajo futuras **39**
7.1. Conclusiones 39
7.2. Líneas de trabajo futuras 41

Bibliografía **43**

1. Introducción

La enfermedad de Parkinson constituye uno de los desafíos sociosanitarios más urgentes de la actualidad. Clínicamente, se caracteriza por una sintomatología motora progresiva, rigidez y bradicinesia, que compromete severamente la autonomía del individuo. Si bien el tratamiento farmacológico es indispensable, existe un consenso médico absoluto sobre el papel de la rehabilitación física continuada como el único mecanismo eficaz para preservar la independencia funcional del paciente a largo plazo.

Esta necesidad es aún más urgente si recordamos que la enfermedad de Parkinson afecta mayoritariamente a personas de edad avanzada. Para este colectivo, la rehabilitación no es solo un ejercicio físico, sino la única herramienta que les permite seguir valiéndose por sí mismos en su día a día.

Sin embargo, la eficacia clínica de la rehabilitación choca frontalmente con una barrera logística y demográfica: la inequidad en el acceso al tratamiento. En un contexto marcado por el envejecimiento poblacional y la dispersión geográfica —fenómenos especialmente agudos en los territorios de la denominada "España Vaciada"—, el modelo tradicional de rehabilitación presencial se vuelve insostenible.

Para un paciente con movilidad reducida que reside en un entorno rural, el desplazamiento diario a un centro hospitalario de referencia no supone únicamente un esfuerzo físico, sino una carga logística y económica que, a menudo, recae sobre las familias. Esta fricción provoca que, en la práctica, muchos pacientes no reciban la terapia necesaria. Ante la saturación de los servicios sanitarios y la dificultad de cobertura en zonas aisladas, surge la necesidad imperiosa de desarrollar soluciones tecnológicas que trasladen la terapia del hospital al domicilio, garantizando la continuidad asistencial sin perder la supervisión facultativa.

En este escenario, los dispositivos móviles se presentan como la herramienta ideal para acercar la terapia al paciente. Su uso se ha extendido enormemente en los últimos años, convirtiéndose en un elemento cotidiano incluso para las personas mayores. Además de su popularidad, ofrecen una ventaja fundamental: su interacción es mucho más intuitiva. Gracias a las pantallas táctiles, se eliminan las barreras que suelen tener los ordenadores convencionales, permitiendo que el paciente acceda a sus ejercicios de forma natural y sencilla.

1.1. Contexto académico

El presente Trabajo de Fin de Grado no se constituye como una iniciativa aislada, sino que se enmarca en una línea de investigación activa de la Universidad de Burgos. El propósito superior de esta investigación es evaluar el impacto de la tecnología en la mejora de la calidad de vida de pacientes con enfermedades neurodegenerativas.

Dentro de este marco, el proyecto se centra en el desarrollo de una infraestructura de telerrehabilitación basada en el uso de dispositivos móviles. El objetivo principal ha sido construir el sistema necesario para capturar y digitalizar los movimientos del paciente a través de la cámara del teléfono o la tableta. Esta herramienta sienta las bases para crear modelos de comparación y evaluación de ejercicios que permitan una telerrehabilitación totalmente autónoma sin la necesidad de un terapeuta.

1.2. Justificación y motivación del proyecto

La elección de este proyecto responde a un doble objetivo: por un lado, aplicar la ingeniería para resolver un problema social real y, por otro, afrontar el reto técnico que supone desarrollar una aplicación completa.

La motivación primaria reside en la vocación de utilidad pública. Frente a la opción de realizar un ejercicio académico teórico, se optó por abordar una problemática tangible: la brecha de acceso a la salud en entornos rurales. El proyecto se concibe como una herramienta de *e-Health* destinada a democratizar la rehabilitación, permitiendo que un paciente de un núcleo rural acceda a la misma calidad de seguimiento que uno urbano. La posibilidad de aportar una solución técnica que impacte directamente en la calidad de vida de un colectivo vulnerable dota al trabajo de una dimensión ética que trasciende la mera programación.

Desde la perspectiva profesional, este TFG se planteó como el escenario idóneo para la transición hacia la ingeniería de software profesional. Al requerir el desarrollo de una solución integral, abarcando desde el diseño de bases de datos relacionales y APIs en el servidor hasta la gestión de hardware multimedia y concurrencia en el dispositivo móvil, el proyecto exigió abandonar la zona de confort académica. Esta complejidad obligó a adoptar un enfoque de autoaprendizaje para dominar tecnologías de desarrollo nativo y arquitecturas modernas demandadas en el mercado laboral actual.

1.3. Objetivos generales

La finalidad última del sistema es eliminar la barrera de la distancia física mediante un puente digital. Se ha diseñado y desarrollado una aplicación móvil nativa que satisface dos funciones críticas:

- **Accesibilidad universal para el paciente:** proveer una herramienta de ejecución de ejercicios guiada por vídeo. Dada la afectación motora y la brecha digital del usuario final, el objetivo prioritario ha sido diseñar una interfaz adaptada a los temblores y la edad, y extremadamente sencilla, garantizando la adherencia al tratamiento.

- **Control asíncrono para el terapeuta:** dotar al profesional sanitario de una plataforma de gestión eficiente. El sistema permite la asignación remota de terapias y, fundamentalmente, la supervisión diferida (asíncrona) mediante la revisión de los vídeos grabados por el paciente, optimizando los tiempos de consulta y permitiendo escalar la atención a un mayor número de usuarios.

1.4. Entregables y repositorio

El resultado tangible de este trabajo es un prototipo funcional de aplicación Android que integra los roles de Administrador, Terapeuta y Paciente, cubriendo el ciclo completo de rehabilitación: prescripción, ejecución, grabación y evaluación.

Para facilitar la auditoría técnica y la reproducibilidad del proyecto, la totalidad del código fuente, junto con la documentación técnica y los scripts de despliegue, se encuentra disponible en el siguiente repositorio de control de versiones:

https://github.com/andrespuentesg/TFG-GII24.36-AndresPuentes

2. Objetivos del Proyecto

El propósito de este Trabajo de Fin de Grado trasciende la mera implementación de software; su finalidad última es aportar una solución de ingeniería efectiva a una problemática sociosanitaria estructural. Para definir el alcance del proyecto y guiar el proceso de desarrollo, se establecieron tres categorías de objetivos: funcionales, técnicos y personales.

2.1. Objetivos funcionales

Definen las metas operativas que el sistema debe satisfacer para resolver las necesidades clínicas y logísticas de los usuarios finales.

- **Digitalización integral del proceso terapéutico:** transformar el flujo de rehabilitación analógico en un ecosistema digital continuo que cubra las tres fases del tratamiento: la prescripción remota de rutinas personalizadas, la ejecución guiada en el domicilio y la evaluación clínica diferida.
- **Centralización del conocimiento terapéutico:** implementar un sistema de gestión de contenidos que permita la creación, clasificación y mantenimiento de una biblioteca digital global de ejercicios. El objetivo es dotar al servicio de rehabilitación de un repositorio de recursos audiovisuales reutilizable y escalable para la configuración de terapias.
- **Garantía de accesibilidad y autenticación delegada:** diseñar una interfaz adaptada a la sintomatología de la enfermedad de Parkinson y eliminar las barreras de acceso tecnológicas. Se busca permitir que el

administrador gestione la sesión en el dispositivo antes de su entrega, liberando al paciente de la gestión de credenciales y ofreciéndole un entorno simplificado basado en interacciones mínimas.

- **Generación de evidencia clínica objetiva:** implementar mecanismos de verificación que obliguen a la captura de vídeo para validar la realización de los ejercicios. Se busca sustituir el reporte subjetivo del paciente por pruebas visuales tangibles, creando un repositorio de datos útil tanto para la corrección inmediata como para la investigación médica futura.

- **Fomento de la adherencia y motivación:** incorporar mecanismos de retroalimentación que permitan al paciente visualizar su evolución histórica y las calificaciones emitidas por el terapeuta, utilizando el refuerzo positivo para mantener el compromiso con la rutina de ejercicios a largo plazo.

- **Optimización de recursos asistenciales:** desacoplar la atención sanitaria de la coincidencia temporal y espacial. El objetivo es permitir que un único terapeuta pueda gestionar y supervisar a un mayor volumen de pacientes de forma eficiente, revisando las sesiones en momentos de baja carga asistencial y optimizando los tiempos de consulta presencial.

- **Gestión estricta de roles y privacidad:** establecer un sistema de control de acceso jerárquico que segmente las funcionalidades y la visualización de datos sensibles según el perfil del usuario (Administrador, Terapeuta o Paciente), garantizando la confidencialidad de la información médica.

2.2. Objetivos técnicos

Este apartado recoge los desafíos de ingeniería y los estándares de calidad software establecidos como meta para la construcción del sistema.

- **Diseño de arquitectura desacoplada y escalable:** implementar un patrón de arquitectura de software que separe estrictamente la lógica de negocio, la capa de presentación y el acceso a datos. El objetivo es evitar el código monolítico para garantizar la mantenibilidad, la testeabilidad y la capacidad del sistema para evolucionar sin requerir reescrituras estructurales.

- **Adopción de paradigmas de interfaz reactivos:** investigar y aplicar estándares modernos de desarrollo de interfaces declarativas. Se busca abandonar los modelos imperativos tradicionales en favor de sistemas reactivos donde la interfaz gráfica sea un reflejo directo e inmutable del estado de la aplicación, minimizando errores de sincronización visual.

- **Abstracción y gestión robusta de hardware:** lograr una integración nativa y estable de la cámara y micrófono dentro del flujo de la aplicación. El reto técnico consiste en gestionar el ciclo de vida de los componentes para prevenir bloqueos, asegurar la sincronización multimedia y garantizar la compatibilidad con la fragmentación del ecosistema de dispositivos móviles.

- **Gestión eficiente del ciclo de vida del dato multimedia:** diseñar mecanismos para la administración del almacenamiento. Dado el alto consumo de recursos del vídeo, el objetivo es dotar al sistema de políticas configurables de retención y purgado, garantizando la sostenibilidad del sistema sin comprometer el historial clínico reciente.

- **Seguridad y protección del dato por diseño:** implementar mecanismos criptográficos para la protección de la información sensible. El objetivo es asegurar que las credenciales de acceso y los registros clínicos se almacenen y transmitan bajo protocolos seguros, cumpliendo con los principios de integridad y confidencialidad exigidos en el software sanitario.

2.3. Objetivos personales

Más allá de los requisitos académicos, este proyecto se concibe como un vector para la maduración profesional del ingeniero.

- **Competencia "Full Stack":** adquirir un dominio transversal del ciclo de vida del desarrollo de aplicaciones. El objetivo es capacitarse tanto en la ingeniería de la interfaz de usuario (Frontend) como en el diseño de la lógica de servidor, APIs y bases de datos (Backend), comprendiendo la integración entre ambas capas.

- **Resolución de problemas en entornos de producción:** trascender el ámbito teórico para enfrentar desafíos propios de la ingeniería real, tales como la tolerancia a fallos del hardware.

- **Especialización en *e-Health*:** desarrollar la habilidad para traducir requisitos clínicos abstractos en especificaciones técnicas precisas. Se busca comprender las implicaciones éticas y funcionales del desarrollo de software para el sector salud, donde la fiabilidad y la accesibilidad son requisitos críticos de seguridad.

- **Aplicación de metodologías ágiles:** ejercitar la capacidad de autogestión y planificación mediante la adopción de marcos de trabajo iterativos, permitiendo una adaptación flexible ante los imprevistos técnicos y una entrega continua de valor funcional.

3. Conceptos Teóricos

A continuación, se detallan los principios teóricos que sustentan la arquitectura del sistema.

3.1. Contexto sanitario

Enfermedad de Parkinson. Se define como un trastorno neurodegenerativo crónico y progresivo que afecta al sistema nervioso central, específicamente a las neuronas encargadas de producir dopamina. Principalmente, esta enfermedad se manifiesta en personas de edad avanzada, aumentando su incidencia significativamente a partir de los 60 años. Desde la perspectiva de la interacción persona-computador, esta patología se caracteriza por presentar síntomas motores severos como temblor en reposo, rigidez muscular y bradicinesia. La comprensión teórica de esta sintomatología es el requisito previo indispensable para el diseño de la interfaz de usuario, imponiendo restricciones estrictas de accesibilidad para garantizar que el software sea operable por personas con control motor fino reducido. [12]

Bradicinesia. Es el término clínico que describe la lentitud en la ejecución de los movimientos voluntarios y la dificultad para iniciarlos. En el contexto del diseño de software para *e-Health*, este síntoma constituye una restricción crítica que obliga a reconsiderar los tiempos de respuesta de la interfaz. Teóricamente, implica que el sistema debe evitar tiempos de espera cortos y eliminar gestos complejos o rápidos, favoreciendo interacciones pausadas y áreas de contacto extendidas para mitigar la frustración del usuario. [13]

Telerrehabilitación. Es la rama de la telemedicina que aplica las Tecnologías de la Información y la Comunicación (TIC) para la provisión de

servicios de rehabilitación a distancia. Este modelo teórico busca descentralizar la atención sanitaria, permitiendo la evaluación, el diagnóstico y el tratamiento de pacientes fuera del entorno hospitalario tradicional. Su objetivo es garantizar la continuidad asistencial, aumentar la adherencia al tratamiento mediante la monitorización remota y democratizar el acceso a terapias especializadas para pacientes con movilidad reducida o residentes en zonas geográficas aisladas.

***e-Health* o Salud Digital.** Se define como la intersección multidisciplinar entre la informática médica, la salud pública y las ciencias de la empresa referida a los servicios sanitarios y la información transmitida a través de Internet y las tecnologías relacionadas. Más allá de una simple implementación técnica, este concepto teórico representa un cambio de paradigma en el modelo asistencial que busca mejorar el acceso, la eficiencia y la calidad de la atención médica mediante la digitalización de procesos. Su aplicación abarca desde la gestión administrativa de historias clínicas electrónicas hasta la provisión de servicios de telemedicina, permitiendo la deslocalización de la asistencia sanitaria. [2]

***m-Health* o Salud Móvil.** Constituye un subsegmento específico dentro de la *e-Health* que se centra en la práctica de la medicina y la salud pública soportada por dispositivos móviles como teléfonos inteligentes, tabletas y tecnologías vestibles. La característica teórica distintiva de este modelo es la ubicuidad y la capacidad de aprovechar los sensores integrados en el hardware del dispositivo, como acelerómetros, giroscopios o cámaras de alta resolución, para la recolección de datos clínicos en tiempo real. En el contexto de la rehabilitación neurológica, la *m-Health* democratiza el acceso al tratamiento al convertir un dispositivo de consumo masivo en una herramienta de monitorización médica continua y accesible desde el domicilio del paciente. [18]

3.2. Ingeniería del software y metodología

Metodología ágil de gestión. Se define como una filosofía de gestión de proyectos que prioriza la flexibilidad, la colaboración y la entrega continua de valor frente a la planificación rígida a largo plazo. En lugar de abordar el desarrollo como un bloque monolítico, se estructura el trabajo en ciclos temporales cortos y repetitivos. Este enfoque permite la inspección constante del progreso y la adaptación rápida a los cambios en los requisitos, asegurando que el producto final se alinee con las necesidades reales del usuario.

Framework. En la ingeniería de software, se define como una estructura conceptual y tecnológica de soporte que sirve de base para la organización y desarrollo de proyectos complejos. A diferencia de una simple librería de código, el *framework* se caracteriza por imponer el principio de inversión de control. Esto significa que no es el código del programador el que llama a las funciones de la herramienta, sino que es el propio entorno el que orquesta el flujo de ejecución general y delega en el código personalizado del desarrollador únicamente los detalles específicos de la lógica de negocio. Su adopción permite estandarizar la arquitectura del software, reutilizar componentes probados y aumentar la productividad al abstraer tareas repetitivas de bajo nivel. [17]

Desarrollo *frontend.* Hace referencia a la ingeniería de la capa de presentación del software. Comprende la implementación de la interfaz gráfica y la lógica de interacción que se ejecuta directamente en el dispositivo del usuario final. Su objetivo teórico es garantizar la usabilidad y la accesibilidad, transformando los datos recibidos del servidor en una experiencia visual comprensible y reactiva con la que el usuario puede interactuar.

Desarrollo *backend.* Constituye la ingeniería de la capa de acceso a datos y lógica de negocio que opera en un servidor remoto, oculta al usuario. Se encarga de procesar las reglas del sistema, gestionar la seguridad, realizar cálculos complejos y asegurar la persistencia de la información en la base de datos. Es el responsable de centralizar la verdad del sistema y servir los datos solicitados por las aplicaciones cliente de manera segura y eficiente.

3.3. Patrones de diseño y arquitectura aplicada

Patrón de arquitectura de software. Este concepto hace referencia a las reglas organizativas que definen la estructura interna del código fuente. Su objetivo fundamental es la separación de responsabilidades, buscando desacoplar la lógica de negocio y la gestión de datos de la interfaz gráfica. Al dividir el sistema en capas independientes con funciones específicas, se facilita el mantenimiento del código, la escalabilidad del proyecto y la realización de pruebas unitarias aisladas.

Patrón de arquitectura *Model-View-ViewModel* (MVVM). Es un patrón de diseño arquitectónico cuyo objetivo fundamental es desacoplar la lógica de presentación y el estado de la interfaz de usuario de la lógica de negocio y los datos. Se estructura en tres componentes diferenciados: el

Modelo, que representa los datos y las reglas de dominio; la Vista, que es un componente pasivo encargado de la representación visual; y el *ViewModel*, que actúa como intermediario gestionando el estado de la aplicación y procesando las interacciones. Su característica teórica distintiva es la ausencia de referencia directa desde el *ViewModel* hacia la Vista, basándose en mecanismos de observación para actualizar la interfaz automáticamente ante cambios en los datos.

Patrón de diseño *Data Access Object* (DAO). Es un patrón estructural utilizado en ingeniería de software para aislar la lógica de negocio de la capa de persistencia. Su propósito teórico es abstraer y encapsular todos los accesos a la fuente de datos. El DAO gestiona la conexión técnica con el mecanismo de almacenamiento (ya sea una base de datos relacional, un archivo local o una API externa) para realizar operaciones de lectura y escritura, exponiendo una interfaz pública agnóstica a la implementación subyacente. Esto permite que el resto de la aplicación opere independientemente de la tecnología de base de datos específica, facilitando la mantenibilidad y la migración de sistemas.

Paradigma de programación declarativa. Se trata de un modelo de programación enfocado en el resultado final visual en lugar de en el flujo de control paso a paso. En este paradigma, el desarrollador describe qué elementos deben aparecer en la pantalla en función del estado actual de la aplicación, delegando en el sistema la responsabilidad de cómo renderizar o actualizar dichos elementos. La interfaz gráfica se concibe como una función directa e inmutable de los datos, actualizándose automáticamente cuando estos varían.

3.4. Conceptos Técnicos de Implementación

Ciclo de vida de la aplicación. Comprende la secuencia de estados por los que transita un proceso de software dentro del sistema operativo, desde su creación hasta su destrucción. Este concepto teórico dicta que una aplicación no es una entidad estática, sino que reacciona a cambios del entorno, como pasar a segundo plano o ser interrumpida. Una correcta gestión de estos estados es crucial para liberar recursos del sistema, como la cámara o la memoria, cuando la aplicación deja de ser visible para el usuario.

Concurrencia estructurada y programación asíncrona. Este concepto computacional se refiere a la capacidad del software para ejecutar tareas múltiples de manera paralela sin bloquear el hilo principal de ejecución.

Es fundamental en entornos con interfaz gráfica para gestionar operaciones de larga duración, como el acceso a redes o el procesamiento de archivos, en un segundo plano. La concurrencia estructurada organiza estas tareas jerárquicamente, garantizando que los procesos secundarios se completen o cancelen de forma ordenada para evitar fugas de memoria.

Endpoint. Constituye el extremo final de un canal de comunicación dentro de una Interfaz de Programación de Aplicaciones (API). En el contexto de los servicios web, un *endpoint* se materializa como una dirección URL específica unida a un método de petición concreto, como GET o POST, que permite a los sistemas clientes interactuar con un recurso determinado del servidor. Desde una perspectiva teórica, representa la interfaz de contrato pública del sistema, definiendo estrictamente la ubicación donde los recursos están disponibles y el formato en el que se debe intercambiar la información, actuando como la puerta de entrada controlada y segura hacia la lógica interna de la aplicación.

Dibujo vectorial. Consiste en una técnica de representación gráfica basada en fórmulas matemáticas y primitivas geométricas, como líneas, curvas y polígonos, en lugar de utilizar matrices de píxeles estáticas. La principal ventaja teórica de este modelo es la independencia de la resolución, lo que permite que los elementos visuales se redimensionen y escalen a cualquier tamaño de pantalla sin sufrir pérdida de nitidez o pixelación.

3.5. Seguridad y privacidad

Aislamiento de aplicaciones o *Sandboxing*. Es un principio fundamental de seguridad informática en sistemas operativos modernos. Establece que cada aplicación se ejecuta en un entorno restringido y privado, con su propio espacio de almacenamiento y memoria asignados. Bajo este modelo teórico, ningún proceso puede acceder a los datos o archivos generados por otra aplicación sin una autorización explícita, garantizando la privacidad y la integridad de la información del usuario. [4]

Cifrado de información. Es la disciplina teórica y matemática responsable de la seguridad de la información mediante la ofuscación de datos. Se define como el procedimiento algorítmico mediante el cual un mensaje legible o texto plano se transforma en un conjunto de caracteres aparentemente aleatorios o texto cifrado. Este proceso garantiza el principio de confidencialidad, asegurando que la información solo pueda ser revertida a su estado original por entidades que posean la clave criptográfica correspondiente, protegiendo

así los datos frente a accesos no autorizados durante su almacenamiento o transmisión.

***Hashing* de cifrado.** Hace referencia a un tipo específico de algoritmo matemático que transforma cualquier bloque de datos arbitrario en una cadena de caracteres de longitud fija y estructura única, denominada *hash* o huella digital. A diferencia del cifrado convencional, su característica teórica fundamental es la unidireccionalidad o irreversibilidad: es computacionalmente inviable recuperar los datos originales a partir del *hash* resultante. Este concepto es el estándar de la industria para el almacenamiento seguro de credenciales y contraseñas, ya que permite verificar la autenticidad de un usuario sin necesidad de guardar su clave real en la base de datos.

4. Técnicas y Herramientas

En este capítulo se describen las tecnologías, lenguajes y herramientas seleccionadas para el desarrollo del sistema. La elección de este stack tecnológico responde a criterios de rendimiento, escalabilidad y adecuación a los requisitos de salud móvil y seguridad descritos anteriormente.

4.1. Herramientas de programación

Entorno de desarrollo Android Studio. Es la herramienta oficial proporcionada por Google para el desarrollo de aplicaciones móviles sobre el sistema operativo Android. Basado en el motor IntelliJ IDEA, este entorno unifica en una sola interfaz todas las herramientas necesarias para la construcción del software, incluyendo un editor de código inteligente, herramientas de depuración avanzada, un emulador de dispositivos y el sistema de construcción Gradle. [8]

Visual Studio Code. Es un editor de código fuente ligero y multiplataforma desarrollado por Microsoft. Se ha seleccionado como el entorno de trabajo principal para el desarrollo del *backend* debido a su arquitectura modular basada en extensiones. Su integración nativa con el intérprete de Python, junto con sus capacidades de depuración remota y gestión de terminales integradas, facilita la implementación ágil de la API.

Lenguaje de programación Kotlin. Se trata de un lenguaje de programación de tipado estático que corre sobre la Máquina Virtual de Java (JVM) y que ha sido adoptado como el estándar oficial para el desarrollo en Android. Su elección se justifica por sus características de seguridad de memoria y por su sintaxis concisa, que permite expresar lógicas complejas con un menor volumen de código. [15]

Python. Es un lenguaje de programación interpretado, multiparadigma y de alto nivel. Ha sido la elección tecnológica para la lógica del servidor debido a su legibilidad y a su vasto ecosistema de bibliotecas especializadas. Su uso permite una implementación rápida de la lógica de negocio y prepara el sistema para una integración fluida con futuros módulos de Inteligencia Artificial. [14]

***Framework* FastAPI.** Es un marco de trabajo moderno y de alto rendimiento para la construcción de APIs con Python. Constituye el núcleo tecnológico del servidor, encargándose de recibir las peticiones de la aplicación móvil, validar los datos y orquestar la lógica de negocio, destacando por su velocidad de ejecución y su capacidad para generar documentación interactiva automática.

Protocolo HTTPS. Constituye la versión segura del protocolo de comunicación HTTP mediante el uso de TLS. Su implementación garantiza la autenticación del servidor, la confidencialidad de los datos médicos en tránsito mediante cifrado y la integridad de la información durante la transmisión entre el cliente y el servidor.

4.2. Bibliotecas

***Toolkit* Jetpack Compose.** Es el conjunto de herramientas moderno recomendado por Google para la construcción de interfaces de usuario nativas. Implementa técnicamente el paradigma de programación declarativa, permitiendo construir la interfaz utilizando exclusivamente código Kotlin, simplificando el desarrollo mediante componentes reutilizables y una gestión reactiva del estado. [9]

Sistema de diseño Material Design 3. Constituye la última iteración del lenguaje de diseño creado por Google. Proporciona una guía exhaustiva de componentes visuales, tipografías y sistemas de color estandarizados que garantizan la coherencia visual y la accesibilidad, asegurando que los elementos interactivos cumplan con los requisitos ergonómicos necesarios para una aplicación de salud. [6]

Biblioteca CameraX. Forma parte de la suite Android Jetpack y es la solución técnica encargada de abstraer la complejidad del hardware de cámara. Su principal valor reside en su capacidad para gestionar el ciclo de vida de la cámara de forma automática, garantizando la estabilidad del sistema y maximizando la compatibilidad con el ecosistema de dispositivos Android.

Reproductor ExoPlayer (Media3). Es un reproductor multimedia a nivel de aplicación, extensible y de código abierto. Permite un control granular sobre la reproducción de vídeo, soportando protocolos de *streaming* adaptativo y ofreciendo una gestión eficiente del *buffering* y la decodificación, elementos críticos para visualizar las sesiones de rehabilitación. [10]

Librería Kotlin Coroutines. Es la herramienta utilizada para implementar el modelo de concurrencia estructurada. Permite transformar operaciones asíncronas complejas en código secuencial y legible. Su uso es vital para mantener la fluidez de la interfaz gráfica, delegando tareas pesadas a hilos de ejecución secundarios de manera eficiente. [7]

Cliente HTTP Retrofit. Es una librería de cliente REST para Android y Java que facilita el consumo de servicios web. Su función es traducir la API HTTP definida en el servidor en interfaces de Kotlin seguras y tipadas, gestionando la serialización y deserialización automática de las respuestas JSON.

Librería de validación de datos Pydantic. Es la herramienta empleada en el entorno de servidor para asegurar la integridad estricta de la información entrante. Actúa como una barrera de control de calidad que verifica que cada petición recibida por la API cumple rigurosamente con el esquema esperado antes de su procesamiento. [3]

Librería de *hashing* Bcrypt. Es una función de derivación de claves criptográficas diseñada para el almacenamiento seguro de contraseñas. Incorpora un coste computacional configurable que hace que el proceso de verificación sea intencionadamente lento para una máquina, protegiendo las credenciales frente a ataques de fuerza bruta. [1]

4.3. Gestión de datos

PostgreSQL. Es un sistema de gestión de bases de datos relacional de código abierto. Actúa como la capa de persistencia definitiva del sistema, garantizando la integridad referencial de los datos y cumpliendo con las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), requisitos indispensables para el almacenamiento seguro de historiales clínicos y registros de usuarios. [16]

Draw.io. Es una aplicación de diagramación técnica multiplataforma y de código abierto. Permite la creación y edición de gráficos vectoriales y modelos visuales estandarizados, como diagramas UML, flujos de procesos

y esquemas de red, proporcionando una interfaz web para el modelado de arquitecturas de software. [11]

4.4. Gestión de proyecto

Sistema de control de versiones Git y GitHub. Git es la herramienta utilizada para registrar los cambios en el código fuente y gestionar ramas de trabajo paralelas. GitHub actúa como la plataforma de alojamiento en la nube, facilitando la copia de seguridad distribuida y la trazabilidad de la evolución del proyecto. [5]

Plataforma de gestión Jira. Es una herramienta de software diseñada para la administración de proyectos y el seguimiento de incidencias. Su arquitectura soporta la implementación de metodologías ágiles, permitiendo la organización de equipos, la asignación de tareas y la monitorización de flujos de trabajo mediante tableros interactivos y reportes de progreso.

5. Aspectos Relevantes del Desarrollo

5.1. Fase de selección de software

El inicio del ciclo de desarrollo se caracterizó por la evaluación técnica de las distintas alternativas disponibles para la implementación del sistema. Tras el análisis de requisitos, se procedió a definir la estrategia arquitectónica, priorizando la viabilidad técnica y el rendimiento sobre la velocidad de desarrollo inicial.

Aunque se consideró el uso de marcos de trabajo multiplataforma o híbridos para facilitar un despliegue rápido en diversos sistemas operativos, el análisis técnico determinó que la solución óptima requería una estrategia de desarrollo nativo basada en el SDK de Android y el lenguaje Kotlin. Esta decisión de ingeniería se fundamentó en cinco criterios técnicos determinantes:

1. **Eficiencia en el acceso al hardware.** Dado que la funcionalidad crítica del sistema depende de la captura intensiva de vídeo y el almacenamiento en disco, la prioridad absoluta fue minimizar la latencia y maximizar la estabilidad. Las soluciones híbridas suelen introducir capas de abstracción o puentes de comunicación que pueden comprometer el rendimiento al acceder a componentes físicos del dispositivo. El desarrollo nativo se seleccionó por ofrecer la integración más directa posible con los sensores de la cámara y el sistema de archivos, garantizando así un control preciso sobre el ciclo de vida del hardware.

2. **Seguridad y estabilidad del lenguaje.** La elección de Kotlin frente a otras alternativas como Java se basó en su condición de estándar

actual en la industria móvil y sus características de seguridad en tiempo de compilación. Específicamente, su sistema de tipos diseñado para la seguridad ante valores nulos permite eliminar una amplia categoría de errores comunes que provocan el cierre inesperado de las aplicaciones. Esta característica fue considerada vital para un entorno de salud, donde la fiabilidad del software es un requisito no funcional prioritario.

3. **Madurez del entorno de ingeniería.** Se valoró la disponibilidad de herramientas avanzadas de depuración y perfilado. El ecosistema nativo, a través de Android Studio, proporciona el conjunto de instrumentos más robusto para la inspección de bases de datos, el análisis de la interfaz gráfica y la monitorización del consumo de memoria y CPU. Disponer de este nivel de observabilidad se consideró indispensable para garantizar la calidad de un software que debe gestionar procesos pesados en segundo plano.

4. **Optimización de la curva de aprendizaje.** Desde una perspectiva de gestión de proyecto, se optó por capitalizar la base de conocimientos previa adquirida durante la formación académica. La decisión estratégica de utilizar una plataforma conocida permitió reducir el tiempo dedicado a los fundamentos básicos del desarrollo. Esto liberó recursos temporales valiosos para ser invertidos en la investigación y el dominio de las tecnologías avanzadas y específicas que demandaba la complejidad del proyecto, como la programación reactiva o la concurrencia.

5. **Prioridad Tablet.** Desde la fase inicial de diseño conceptual se estableció una distinción clara respecto al dispositivo de despliegue para el módulo del paciente. A diferencia de la aplicación del terapeuta, diseñada para un uso flexible en teléfonos inteligentes, la interfaz del paciente se concibió y optimizó primariamente para su ejecución en dispositivos tipo *tablet*. Esta decisión estratégica de diseño respondió a tres factores ergonómicos y funcionales ineludibles:

   - **Maximización del área de interacción:** la sintomatología motora propia de la enfermedad, caracterizada por temblores y rigidez, dificulta la precisión en el toque sobre pantallas reducidas. El factor de forma de una *tablet* proporciona una superficie táctil significativamente mayor. Esto permitió planificar desde el inicio la implementación de elementos de interfaz de gran tamaño y márgenes de error generosos, reduciendo la frustración del usuario durante la navegación.

- **Visualización de contenidos terapéuticos:** dado el perfil demográfico de los usuarios, frecuentemente asociado a una edad avanzada y presbicia, el tamaño de pantalla extendido resulta indispensable. Este soporte permite la reproducción de los vídeos guía con un nivel de detalle y escala suficientes para que el paciente pueda imitar los movimientos correctamente desde cierta distancia, sin forzar la vista ni necesidad de manipular el dispositivo constantemente.
- **Estabilidad en el uso manos libres:** la naturaleza de la terapia física requiere que el paciente disponga de libertad de movimiento corporal total. El formato *tablet* favorece un uso estacionario, permitiendo apoyar el dispositivo de forma estable en una mesa o soporte fijo para seguir las instrucciones visuales. Esto contrasta con la ergonomía del teléfono móvil, cuyo uso habitual en mano interferiría con la correcta ejecución de la rutina de ejercicios.

5.2. Fase de definición de requisitos y funcionalidades

La fase de definición de requisitos constituyó uno de los pilares fundamentales del proyecto, consumiendo una parte significativa del esfuerzo de planificación. Dada la complejidad de las interacciones entre los distintos perfiles y la criticidad de los datos médicos, se descartó un enfoque lineal en favor de una metodología de refinamiento iterativo.

Este proceso no se limitó a una simple enumeración de deseos, sino que implicó la reescritura constante de los casos de uso y la validación continua de las reglas de negocio. El objetivo prioritario fue alcanzar una definición del sistema lo suficientemente sólida y granular como para cubrir todos los casos bordes antes de escribir el código, facilitando así el diseño posterior de un modelo relacional robusto y libre de inconsistencias estructurales.

Definición de actores y alcance funcional

Tras múltiples ciclos de análisis, se estructuró la lógica de la aplicación en torno a tres actores claramente diferenciados, cada uno con un ámbito de responsabilidad estanco y funciones específicas.

1. **Rol de Administrador:** este perfil técnico actúa como el garante de la seguridad y la organización del sistema. Su funcionalidad se ha

diseñado para ser invisible al proceso clínico, pero indispensable para el funcionamiento de la plataforma.

- **Gestión del ciclo de vida de usuarios:** responsabilidad exclusiva sobre las operaciones de alta, baja y modificación de las credenciales de acceso, garantizando que solo personal autorizado acceda al sistema.
- **Orquestación de relaciones:** Gestión de la lógica de asignación entre profesionales y usuarios. El administrador es el único facultado para establecer los vínculos o pares Terapeuta-Paciente, definiendo qué profesional tiene permisos de lectura y escritura sobre los datos de qué paciente.

2. **Rol de Terapeuta:** es el actor que dota de contenido al sistema. Sus requisitos funcionales se diseñaron con una estructura jerárquica para permitir una reutilización eficiente del trabajo clínico:

   - **Gestión de Ejercicios:** capacidad para crear, editar y eliminar ejercicios base, subiendo los vídeos de ejemplo y definiendo los parámetros de ejecución.
   - **Gestión de Rutinas:** funcionalidad para componer conjuntos lógicos de ejercicios (rutinas) que pueden ser reutilizados. Esto evita que el terapeuta tenga que seleccionar los ejercicios uno a uno cada vez.
   - **Gestión de Sesiones:** es la materialización temporal de la terapia. El sistema permite asignar una rutina específica a un paciente concreto en una fecha y hora determinadas. Esta distinción entre Rutina y Sesión fue una conclusión clave del análisis para permitir la planificación a largo plazo.

3. **Rol de Paciente:** el diseño funcional de este actor se rige por el principio de mínima interacción necesaria. Su interfaz se ha despojado de cualquier gestión administrativa para centrarse exclusivamente en la terapia.

   - **Realización de sesiones programadas:** acceso directo y simplificado a la sesión del día, con una guía paso a paso para la reproducción del ejemplo y la grabación de la respuesta.
   - **Visualización del progreso:** funcionalidad para consultar el historial de calificaciones y comentarios emitidos por el terapeuta. Este requisito se introdujo con una finalidad psicológica: fomentar

la motivación y la autoestima del paciente al permitirle visualizar tangiblemente su evolución y el cumplimiento de sus objetivos.

Consolidación y diseño del modelo

La exhaustividad aplicada en esta fase de análisis tuvo un impacto directo en la calidad de la implementación posterior. La identificación temprana de entidades complejas, como la diferenciación entre rutina y sesión o la relación N:M entre terapeutas y pacientes, permitió diseñar un esquema de base de datos relacional altamente normalizado desde el inicio. Esta inversión de tiempo en la fase de requisitos minimizó drásticamente la deuda técnica y la necesidad de refactorización durante el desarrollo del código.

5.3. Fase de diseño e implementación

Una vez cerrados los requisitos funcionales, se abordó la construcción de la capa de persistencia. Lejos de proceder a una implementación inmediata, se priorizó una etapa de análisis estructural exhaustivo bajo la premisa de que la solidez de la lógica de negocio depende directamente de la calidad del modelo de datos subyacente.

El proceso de ingeniería de datos se ejecutó siguiendo una metodología secuencial dividida en dos etapas diferenciadas:

Etapa 1: en primera instancia se elaboró el diseño teórico del sistema mediante diagramas Entidad-Relación. Durante esta etapa el esfuerzo se focalizó en la identificación precisa de las entidades y la definición estricta de las cardinalidades entre ellas. Se aplicaron reglas de normalización para eliminar redundancias y se diseñaron las restricciones de integridad referencial sobre el plano teórico, asegurando la coherencia estructural de la información antes de escribir cualquier línea de código.

Etapa 2: se procedió a la materialización del esquema lógico en el Sistema Gestor de Bases de Datos. El diseño teórico se tradujo a sentencias SQL y Lenguaje de Definición de Datos, generando las tablas, claves foráneas e índices necesarios para optimizar el rendimiento de las consultas que realizaría la aplicación.

Esquema Relacional del Sistema

Como resultado de este proceso de análisis, se obtuvo un modelo relacional robusto capaz de soportar la complejidad de las interacciones entre los

distintos roles y la gestión temporal de las sesiones de terapia. A continuación, se presenta el diagrama final implementado en el sistema:

Selección del Motor de Base de Datos: PostgreSQL

Para la gestión y persistencia de la información se seleccionó el motor PostgreSQL. Esta decisión tecnológica no fue accesoria, sino que se fundamentó en dos criterios estratégicos alineados con la naturaleza del proyecto:

- **Fiabilidad y cumplimiento ACID.** Dada la criticidad de los datos gestionados, correspondientes a historiales de salud y seguimiento terapéutico, la integridad de la información se estableció como un requisito no negociable. PostgreSQL fue seleccionado por su reconocida robustez y su estricto cumplimiento de las propiedades ACID, las cuales garantizan la Atomicidad, Consistencia, Aislamiento y Durabilidad de las transacciones. Esto asegura que, ante cualquier fallo del sistema o error durante una operación de escritura, la base de datos nunca quede en un estado inconsistente, protegiendo la validez de los registros médicos.
- **Eficiencia operativa y capitalización de conocimiento.** Desde la perspectiva de la gestión de recursos del proyecto, se valoró la experiencia técnica previa adquirida con esta tecnología durante la formación académica. La decisión de utilizar un motor consolidado y conocido permitió agilizar drásticamente la fase de configuración de la infraestructura. Esta estrategia de optimización liberó tiempo valioso del cronograma de desarrollo, permitiendo redirigir ese esfuerzo hacia la implementación de la lógica compleja de la aplicación móvil y el procesamiento multimedia.

5.4. Fase de desarrollo de la API REST

El desarrollo de la infraestructura de servidor marcó la transición del proyecto hacia una arquitectura distribuida completa. Esta fase implicó la ingeniería del *Backend* con el objetivo de establecer un canal de comunicación seguro y estandarizado entre la base de datos relacional y la aplicación cliente Android.

Para materializar esta conexión, se diseñó e implementó una interfaz de programación de aplicaciones basada en el estilo arquitectónico REST.

Este componente actúa como la única puerta de entrada al sistema, centralizando la lógica de negocio y garantizando la integridad de los datos independientemente del dispositivo que realice la petición.

Selección de FastAPI

La elección de FastAPI como marco de trabajo para el servidor se fundamentó en criterios de eficiencia operativa y calidad del software resultante:

- **Optimización de la curva de aprendizaje:** al aprovechar el dominio previo del lenguaje Python, se pudo focalizar el esfuerzo de ingeniería en la correcta implementación de los patrones REST y la seguridad, en lugar de en la sintaxis de un nuevo lenguaje.
- **Documentación viva y estandarizada:** la capacidad del *framework* para generar automáticamente documentación interactiva basada en el estándar OpenAPI resultó crítica para el desarrollo. Permitió la validación inmediata de los puntos de acceso o *endpoints* y facilitó la integración con el cliente móvil sin necesidad de mantener documentación manual externa.

Encapsulamiento de la Lógica de Dominio

La API no se concibió como una simple pasarela de acceso a datos, sino como el garante de las reglas de negocio del sistema. Se programaron controles estrictos en el servidor para asegurar la consistencia del modelo:

- **Aislamiento contextual de datos:** se implementaron filtros de seguridad a nivel de consulta para garantizar que las peticiones de un paciente solo retornen información perteneciente a su propio historial clínico, previniendo fugas de información cruzada entre usuarios.
- **Validación de estados:** el servidor gestiona la lógica de las transacciones complejas, impidiendo situaciones inconsistentes como la duplicación de rutinas en una misma fecha o la alteración de registros médicos ya consolidados.

Seguridad y Calidad del Software

Dada la sensibilidad de la información gestionada, se aplicaron principios de seguridad desde el diseño. Se integró la librería Bcrypt para el *hashing*

unidireccional de contraseñas, asegurando que las credenciales de acceso nunca se almacenen en texto plano.

Adicionalmente, se desarrolló una batería de pruebas de integración automatizadas. Estos tests verifican de forma continua que las restricciones de privacidad y las reglas de negocio se mantienen vigentes ante cualquier modificación del código fuente.

5.5. Fase de creación de aplicación móvil

Tras el despliegue de la API, se abordó la construcción de la aplicación móvil. Ante la divergencia radical de necesidades entre los perfiles de gestión y los pacientes, se optó por una arquitectura de interfaz segregada. Técnicamente, la aplicación contiene tres flujos de navegación independientes que se activan dinámicamente según el rol del usuario autenticado, comportándose como tres herramientas especializadas dentro de un mismo ejecutable.

Para materializar esta segregación, el diseño de las interfaces se fundamentó en principios de usabilidad y experiencia de usuario adaptados a las competencias digitales de cada arquetipo. Se adoptó el sistema de diseño Material Design 3 para garantizar la consistencia visual y la familiaridad de los componentes interactivos. La estrategia de navegación se definió bajo dos paradigmas diferenciados. Por un lado, para el perfil de paciente se priorizó la accesibilidad y la linealidad, implementando flujos secuenciales guiados y elementos de interacción de gran tamaño que minimizan la carga cognitiva y facilitan el uso a personas con posibles limitaciones motoras o tecnológicas. Por otro lado, para el perfil terapéutico y administrativo se optó por una estructura de navegación orientada a la productividad, permitiendo el acceso rápido a la gestión de expedientes y rutinas.

Módulo de Administración: Integridad Referencial

El módulo de administración se diseñó priorizando la seguridad operativa sobre la estética. Su función es la gestión del ciclo de vida de las identidades mediante operaciones CRUD.

El principal desafío técnico residió en asegurar la integridad referencial desde la interfaz de usuario. El sistema implementa validaciones previas que impiden la creación de registros inconsistentes, como la asignación de pacientes a terapeutas inexistentes, actuando como una primera línea de defensa antes de enviar la petición al servidor.

Módulo de Terapia: Eficiencia y Asincronía

La interfaz del terapeuta requirió una ingeniería enfocada en la productividad y el manejo de grandes volúmenes de datos.

- **Renderizado eficiente:** se utilizaron componentes de lista perezosa para la visualización de las carteras de pacientes, optimizando el consumo de memoria al cargar los elementos bajo demanda.
- **Evaluación multimedia integrada:** Se integró un reproductor de vídeo interno que permite la visualización y calificación de los ejercicios sin salir de la aplicación. Esto cierra el ciclo de telerrehabilitación, habilitando la evaluación asíncrona.

Módulo de Paciente: Accesibilidad y Automatización

El diseño para el paciente representó el mayor reto de usabilidad. Se descartaron los patrones de navegación estándar de Android en favor de una interfaz lineal y adaptada.

- Se implementó un flujo de "Manos Libres" donde la aplicación orquesta la sesión. El sistema reproduce automáticamente el vídeo de ejemplo y gestiona el inicio y fin de la grabación, minimizando la necesidad de interacción física táctil, lo cual es crítico para usuarios con temblores.
- Adicionalmente, se integró un mecanismo de reintento inmediato que permite al paciente descartar la grabación y reiniciar el ejercicio ante cualquier dificultad. Esto garantiza la validez del material clínico y otorga flexibilidad al usuario para corregir su ejecución sin alterar el flujo automatizado de la sesión.

Evolución y Refactorización del Motor de Sesiones

Durante la fase de integración se identificaron limitaciones en el diseño inicial que obligaron a evolucionar el software.

- **Adaptación del modelo de datos:** fue necesario ampliar el esquema de base de datos y los *endpoints* de la API para incluir metadatos de estado más granulares, permitiendo una gestión precisa del progreso del paciente.

- **Concurrencia y actualización optimista:** Para resolver problemas de latencia en la actualización del estado de los ejercicios (completado/pendiente), se implementó un patrón de actualización optimista en la interfaz. La aplicación refleja el cambio de estado inmediatamente para el usuario, mientras sincroniza la información con el servidor en segundo plano, mejorando la percepción de fluidez y evitando inconsistencias visuales.

5.6. Fase de gestión de vídeos

El módulo de gestión de vídeo constituyó el componente de mayor complejidad técnica, requiriendo orquestar la captura, reproducción y persistencia de archivos pesados bajo restricciones de rendimiento móvil.

Motor de Reproducción Nativo y Control de Flujo

Se desestimó el uso de reproductores externos o vistas web con el fin de optimizar la usabilidad y reducir la carga operativa del paciente. En su lugar, se implementó una solución nativa basada en la biblioteca Media3/ExoPlayer.

Esta decisión permitió obtener un control programático total sobre el ciclo de vida de la reproducción. Mediante la monitorización de eventos de estado, el sistema es capaz de detectar la finalización del vídeo de ejemplo y transicionar automáticamente a la pantalla de grabación, eliminando la necesidad de intervención manual por parte del usuario.

Estrategia de Persistencia y Abstracción

Para el almacenamiento de los vídeos generados, se diseñó una arquitectura preparada para el escalado a la nube, aunque implementada localmente para la fase de prototipo. Se aplicó un patrón de diseño que abstrae la fuente de datos, desacoplando la lógica de la aplicación de la ubicación física de los archivos. Esto permite que la capa de presentación ignore si el recurso es local o remoto, facilitando una futura migración a servicios de almacenamiento en la nube sin requerir la refactorización del código cliente.

Tolerancia a Fallos e Integridad de Archivos

La integración con el hardware de cámara presentó desafíos de estabilidad. Para mitigar el riesgo de corrupción de archivos debido a interrupciones

inesperadas del sistema operativo, se desarrolló una capa de programación defensiva.

El sistema incorpora rutinas de verificación post-procesado que analizan la integridad de los archivos de vídeo generados antes de darlos por válidos. Si se detecta un archivo corrupto o vacío, el sistema lo descarta de forma controlada y gestiona el error, impidiendo que se persistan datos inválidos en la base de datos clínica y asegurando la robustez del repositorio multimedia.

6. Trabajos Relacionados

El desarrollo de este proyecto no parte de cero, sino que se integra dentro de una línea de investigación consolidada en la Universidad de Burgos orientada a la aplicación de tecnologías informáticas para la mejora de la calidad de vida en enfermedades neurodegenerativas. A continuación, se analizan los antecedentes académicos directos que han servido como referencia conceptual y técnica, así como soluciones comerciales que comparten principios de diseño.

6.1. Antecedentes Académicos: TFM

Esta primera categoría engloba las tesis de máster que establecieron las bases tecnológicas del sistema, centrándose en la visión artificial y la arquitectura del servidor.

Detección de poses con Detectron2 y comparación de vídeos

- **Referencia:** Ramírez Sanz, J. M. (TFM).
- **Enlace:** https://github.com/Josemi/TFM-FIS-IA

Este Trabajo de Fin de Máster constituye el cimiento tecnológico sobre el que se asienta la capacidad de análisis del proyecto global. La investigación partió de la necesidad de mitigar los efectos de la despoblación rural y la saturación de los centros de salud mediante la automatización de la terapia física.

El núcleo del estudio consistió en una revisión exhaustiva del estado del arte en materia de visión artificial para seleccionar la arquitectura más idónea en la tarea de estimación de pose humana. Tras evaluar diversas alternativas, la investigación se decantó por la implementación de Detectron2, una plataforma de detección de objetos de última generación.

El aporte técnico principal de este trabajo fue el desarrollo de un sistema de extracción de características capaz de inferir la estructura corporal o esqueleto de una persona a partir de una imagen plana. Asimismo, se implementaron los primeros algoritmos de lógica comparativa que permiten cotejar matemáticamente la posición del paciente frente a la del terapeuta. Esta funcionalidad es la que habilita teóricamente la retroalimentación automática, permitiendo al sistema evaluar si una postura es correcta sin intervención humana directa.

Sinergia y Dependencia Tecnológica: La relación entre este TFM y el presente proyecto es de naturaleza jerárquica y complementaria. El trabajo de Ramírez Sanz validó la viabilidad técnica de la solución: demostró que es posible utilizar inteligencia artificial para supervisar ejercicios de rehabilitación. Por su parte, el presente Trabajo de Fin de Grado materializa el canal de distribución de dicha tecnología. Mientras que el TFM se centró en el modelo matemático y el procesamiento de imagen en el servidor, este proyecto se ha encargado de construir la infraestructura de captura en el extremo del cliente. La aplicación móvil desarrollada es el vehículo indispensable para llevar los algoritmos de visión artificial validados por Ramírez Sanz hasta el domicilio del paciente, cerrando así la brecha entre la investigación teórica en IA y su aplicación práctica en un entorno doméstico real.

Creación de la infraestructura necesaria para la telerrehabilitación y análisis online

- **Referencia:** Garrido Labrador, J. L. (TFM).
- **Enlace:** https://github.com/jlgarridol/FIS-FBIS

Este proyecto de máster abordó el desafío de la telerrehabilitación desde una perspectiva de arquitectura de sistemas y escalabilidad. Su foco principal no fue el algoritmo específico de análisis, sino el diseño de una infraestructura *Full-Stack* robusta capaz de soportar la carga masiva de datos multimedia inherente a un despliegue poblacional.

La investigación identificó que, para que la telemedicina sea viable en zonas rurales ("España Vaciada") y reduzca costes reales, el sistema no puede depender exclusivamente de la revisión manual asíncrona, ya que esto crea un cuello de botella en el facultativo. Por ello, se propuso una arquitectura basada en tecnologías de Big Data y Redes Neuronales Profundas.

El aporte fundamental de este trabajo fue la implementación de flujos de ingestión de datos capaces de gestionar la subida simultánea de vídeos por parte de múltiples pacientes. Asimismo, se integraron los modelos de Inteligencia Artificial para la identificación de esqueletos (*skeleton tracking*) dentro de un *pipeline* de procesamiento automatizado, sentando las bases para que la evaluación clínica pudiera realizarse de forma desatendida mediante el análisis computacional.

Complementariedad Arquitectónica: La vinculación entre este TFM y el presente Trabajo de Fin de Grado es estructural. Se establece una relación de Cliente-Infraestructura. El trabajo de Garrido Labrador definió y construyó el "lado del servidor" (*Backend*) a gran escala, preparado para recibir y procesar terabytes de información médica. En contrapartida, el presente proyecto constituye la implementación del "terminal de usuario" (*Frontend* Móvil). La aplicación Android desarrollada en este TFG es el componente encargado de digitalizar al paciente y transmitir la información hacia la infraestructura de Big Data diseñada por Garrido, actuando como el sensor distribuido que alimenta a todo el ecosistema de análisis.

6.2. Antecedentes Académicos: TFG

Este bloque agrupa los trabajos de grado que, partiendo de la infraestructura anterior, desarrollaron módulos específicos para la evaluación del desempeño del paciente.

Evaluación de Ejercicios de Rehabilitación

- **Referencia:** Núñez Calvo, L. (2022).
- **Enlace:** https://github.com/2021-22-TFG-jul/TFG-Evaluacion-Ejercicios-Rehabilitacion

Este trabajo de investigación previo abordó la problemática de la rehabilitación desde una perspectiva algorítmica y de procesamiento de imagen.

Su objetivo principal se centró en la segmentación temporal automática de ejercicios terapéuticos dentro de secuencias de vídeo de larga duración.

La autora partió de la premisa de que la enfermedad de Parkinson, caracterizada por la pérdida de coordinación y control motor, requiere un monitoreo preciso de la ejecución de las terapias. Para ello, su estudio se basó en el análisis de esqueletos digitales (*skeletonization*) extraídos de los fotogramas del vídeo, una técnica heredada de trabajos de máster anteriores (Ramírez Sanz y Garrido Labrador).

La propuesta técnica de Núñez Calvo consistió en desarrollar un sistema capaz de comparar los patrones de movimiento del terapeuta (patrón oro) con los del paciente. Mediante algoritmos de coincidencia, el sistema lograba localizar e identificar automáticamente en qué momento exacto de la grabación del paciente se ejecutaba el ejercicio prescrito por el terapeuta, permitiendo evaluar la precisión del movimiento. El resultado final de esta investigación se materializó en una aplicación de escritorio orientada al análisis y visualización de estos recortes.

Relación y Diferenciación con el Presente Proyecto: El análisis del trabajo de Núñez Calvo resulta fundamental para delimitar el alcance de este Trabajo de Fin de Grado. Mientras que su investigación se focalizó en el post-procesado y análisis algorítmico de los vídeos en un entorno de escritorio, el presente proyecto se centra en la etapa previa: la adquisición y digitalización del dato en un entorno móvil y doméstico. La infraestructura desarrollada en este TFG actúa como el proveedor de datos necesario para sistemas como el propuesto por Núñez Calvo. Al garantizar que el paciente graba el ejercicio correctamente en su domicilio y que dicho vídeo se transmite de forma estructurada e íntegra al servidor, se generan los insumos brutos indispensables para que, en etapas futuras, algoritmos de segmentación y evaluación automática puedan procesar dicha información. Por tanto, ambos trabajos son complementarios dentro de la arquitectura global de telerrehabilitación: uno resuelve la logística de captura (Mobile) y el otro la lógica de evaluación (Desktop/Algorítmica).

Detección de ejercicios en vídeos de rehabilitación

- **Referencia:** Espinosa Lafuente, L. A. (2024).
- **Enlace:** https://github.com/2023-24-TFG-jul/Deteccion-de-ejercicios-en-videos-de-rehabilitacion

Este proyecto representa la continuación directa y la evolución funcional de la línea de investigación iniciada por Núñez Calvo. Si el trabajo anterior se centraba en la segmentación temporal (identificar el intervalo de tiempo del ejercicio), la investigación de Espinosa Lafuente avanzó hacia la valoración cualitativa de la ejecución motora.

El núcleo de su contribución residió en la automatización del proceso de evaluación clínica. Partiendo de los módulos de extracción de esqueletos y recorte temporal desarrollados en los trabajos precedentes, el autor implementó algoritmos capaces de comparar biomecánicamente el vídeo del paciente frente al del profesional.

El principal desafío técnico resuelto en este estudio fue la gestión de la variabilidad temporal. Dado que los pacientes con enfermedad de Parkinson suelen ejecutar los movimientos con bradicinesia (lentitud) o ritmos irregulares, el sistema fue diseñado para comparar eficazmente dos secuencias de vídeo aunque estas tuvieran duraciones distintas. El resultado de este procesamiento es la generación automática de una puntuación objetiva sobre el desempeño del usuario, visualizable a través de una interfaz web desarrollada para tal fin.

Relación Funcional: La relación entre el trabajo de Espinosa Lafuente y el actual es de interdependencia funcional. El sistema de evaluación automática propuesto por Espinosa requiere una entrada de datos constante y estandarizada para funcionar. Mientras que su TFG se focalizó en la algoritmia de comparación y la visualización web de resultados, el presente proyecto resuelve la problemática de la adquisición remota del dato. La aplicación móvil aquí desarrollada actúa como el eslabón logístico necesario para capturar el vídeo en el domicilio del paciente, garantizar su integridad y enviarlo al servidor. En un escenario de despliegue real, la infraestructura móvil de este TFG sería la encargada de alimentar con vídeos validados al motor de puntuación desarrollado por Espinosa Lafuente.

6.3. Soluciones Comerciales de Referencia

Más allá del ámbito académico, existen aplicaciones en el mercado que han abordado el diseño de interfaces para poblaciones con necesidades especiales o que implementan flujos de navegación lineal ("modo túnel") similares a los propuestos en este proyecto para el perfil de paciente.

Big Launcher

Esta aplicación es un referente en accesibilidad móvil para personas mayores o con problemas de visión. Su filosofía de diseño coincide plenamente con la implementada en el perfil de paciente de nuestro proyecto:

- **Interfaz simplificada:** Sustituye la complejidad estándar de Android por una cuadrícula de botones grandes y de alto contraste.
- **Navegación lineal y jerarquía plana:** Elimina menús ocultos o gestos complejos, priorizando que cada acción tenga un botón visible y único.
- **Prevención de errores:** Bloquea funciones críticas para evitar borrados accidentales o desconfiguraciones, un principio que hemos aplicado en el flujo de grabación "manos libres" para evitar que el paciente interrumpa la sesión involuntariamente.

Lumosity

Aunque es una aplicación de entrenamiento cognitivo, su diseño de experiencia de usuario (UX) es un modelo de navegación guiada o "modo túnel".

- **Flujo "Túnel":** Al iniciar el entrenamiento diario, la app toma el control y guía al usuario de un ejercicio al siguiente sin devolverlo al menú principal hasta terminar la sesión. Este es exactamente el comportamiento implementado en nuestra "Sesión Guiada", donde el paciente es llevado automáticamente de la visualización del ejemplo a la grabación y luego al siguiente ejercicio.
- **Feedback inmediato y simple:** Utiliza indicadores visuales claros y grandes para confirmar el éxito o la finalización de una tarea, reduciendo la carga cognitiva del usuario, un aspecto vital para pacientes con enfermedades neurodegenerativas.

Kaia Health

Es una aplicación de terapia digital para el manejo del dolor (como dolor de espalda o EPOC) que utiliza visión por computadora en el móvil, similar al objetivo final de nuestra línea de investigación.

- **Asistente de Movimiento:** Utiliza la cámara del móvil para rastrear puntos del cuerpo y dar *feedback*, validando el uso del *smartphone* como herramienta de captura biométrica en el hogar, tal como proponemos.
- **Interacción sin manos:** Diseñada para que el usuario coloque el móvil en una posición fija y siga las instrucciones verbales y visuales, minimizando la interacción táctil durante el ejercicio, lo cual valida nuestro enfoque de "manos libres" para pacientes con Parkinson.

7. Conclusiones y Líneas de trabajo futuras

La finalización de este Trabajo de Fin de Grado representa la consecución de un hito tecnológico clave dentro de la línea de investigación propuesta por la Universidad de Burgos. Más allá del cumplimiento académico, el proyecto ha logrado diseñar, desarrollar y validar un sistema integral de telerrehabilitación que no solo satisface los requisitos funcionales iniciales, sino que aporta soluciones de ingeniería robustas a problemas complejos de accesibilidad y captura de datos multimedia.

7.1. Conclusiones

Tras el proceso de análisis, diseño y validación del sistema, se han extraído las siguientes conclusiones clasificadas por su área de impacto:

Perspectiva Funcional y Clínica

El sistema desarrollado ha validado la viabilidad técnica de trasladar la rehabilitación neurológica al entorno domiciliario sin perder rigor en el seguimiento.

- **Cambio de Paradigma (Modelo Asíncrono):** La aplicación rompe eficazmente la barrera geográfica y temporal. Al implementar un modelo de comunicación asíncrona, se soluciona el cuello de botella de la saturación en consultas presenciales y se democratiza el acceso a la terapia para pacientes del entorno rural.

- **Digitalización "End-to-End":** Se ha logrado informatizar con éxito todo el flujo de trabajo: desde la prescripción de rutinas hasta la evaluación cuantitativa y cualitativa. Esto transforma la rehabilitación de un evento episódico a un proceso continuo y trazable.

Accesibilidad y Experiencia de Usuario (UX)

Uno de los mayores retos de ingeniería fue adaptar la tecnología a un perfil de usuario con limitaciones motoras severas y baja alfabetización digital.

- **Viabilidad del Diseño Inclusivo:** El uso de Jetpack Compose ha permitido implementar una interfaz no convencional, alejada de los estándares densos de Android. La priorización de elementos de gran tamaño y altos contrastes ha validado la hipótesis de que una interfaz adaptada reduce drásticamente los errores de interacción.
- **Gestión de la Carga Cognitiva:** La implementación de flujos guiados y modo "Manos Libres" ha demostrado ser crítica. Al delegar en el software la gestión de los tiempos y el control de la cámara, se libera al paciente de la fricción tecnológica.

Ingeniería del Software y Calidad Técnica

El proyecto ha superado desafíos de bajo nivel para garantizar la estabilidad y la calidad del dato bajo patrones de diseño modernos.

- **Integridad Multimedia:** La integración de la librería CameraX ha garantizado que el activo más valioso del sistema —el vídeo— se genere y persista siempre de forma íntegra, gestionando correctamente el almacenamiento local.
- **Arquitectura Robusta:** La adopción del patrón MVVM y el patrón Repository, siguiendo las guías oficiales de Android, asegura un código modular. Aunque el prototipo utiliza almacenamiento local, la abstracción de fuentes de datos deja al sistema preparado para conectarse a un servidor remoto con cambios mínimos.
- **Eficiencia de Ejecución:** El uso de la programación asíncrona con Kotlin Coroutines ha permitido mantener un rendimiento fluido (60 fps) gestionando simultáneamente la previsualización de cámara y la escritura de archivos.

Aportación a la Investigación (Valor del Dato)

Este proyecto se constituye como el motor de adquisición de datos para las siguientes fases de la investigación académica de la Universidad de Burgos.

- **Homogeneización del Dataset:** La aplicación estandariza técnicamente el formato, la resolución y el encuadre de las grabaciones, proporcionando un conjunto de datos homogéneo.
- **Habilitador de IA:** Esta estandarización es el prerrequisito indispensable para que futuros algoritmos de Inteligencia Artificial puedan entrenarse con fiabilidad para la detección de poses y métricas biomecánicas.

7.2. Líneas de trabajo futuras

El desarrollo actual constituye una base sólida y funcional que cumple con los objetivos del TFG. No obstante, se han identificado las siguientes vías de evolución técnica para futuras iteraciones del software:

1. **Migración a Arquitectura Distribuida (Cloud):** Actualmente, la persistencia de datos se realiza de forma local (*Edge Storage*). El siguiente paso lógico es la transición hacia una arquitectura cliente-servidor completa bajo el estilo REST. *Objetivo:* Centralizar la información en la nube utilizando FastAPI y PostgreSQL para permitir al terapeuta evaluar las sesiones desde una estación de trabajo hospitalaria.
2. **Tele-rehabilitación Síncrona (Control Remoto):** Se ha detectado la necesidad de asistir a pacientes con deterioro cognitivo severo. *Objetivo:* Implementar comunicación en tiempo real que habilite una "Sesión Asistida", donde el terapeuta tome el control remoto de la aplicación del paciente para lanzar los vídeos y gestionar la grabación.
3. **Integración de Visión por Computador e IA:** Esta línea representa la convergencia final con el proyecto de investigación macro. *Objetivo:* Integrar librerías de Visión por Computador para automatizar el análisis biomecánico, detectando métricas objetivas (velocidad, amplitud angular) y ofreciendo *feedback* correctivo en tiempo real.
4. **Estrategias de Adherencia (Gamificación):** La rehabilitación exige una constancia que frecuentemente genera fatiga psicológica.

Objetivo: Implementar mecánicas de ludificación (recompensas, medallas, rachas) para aumentar la motivación intrínseca y las tasas de adherencia terapéutica.

5. **Ciberseguridad y Cumplimiento Normativo:** Al migrar los datos a la nube, la protección del dato médico se vuelve crítica. *Objetivo:* Implementar protocolos de Cifrado de Extremo a Extremo (E2EE) y garantizar el cumplimiento estricto del RGPD para la transmisión de vídeos e información sensible entre el paciente y el terapeuta.

Bibliografía

[1] Bcrypt-generator. Bcrypt Generator & Tool. https://bcrypt-generator.com/, 2026. Herramienta de verificación de hashing.

[2] Biblioteca UOC. eHealth: la transformación del modelo asistencial. https://biblioteca.uoc.edu/es/biblioguias/biblioguia/eHealth-la-transformacion-del-modelo-asistencial, 2026. Análisis sobre la digitalización de la salud.

[3] Samuel Colvin. Pydantic v2 Documentation. https://docs.pydantic.dev/latest/, 2026. Validación de datos en el servidor.

[4] ESED. Sandboxing: qué es, cómo funciona y protección ante malware. https://www.esedsl.com/blog/sandboxing-que-es-como-funciona-y-como-te-protege-ante-malware, 2026. Conceptos de seguridad y aislamiento de procesos.

[5] Git. Git: Sistema de control de versiones distribuido. https://git-scm.com/, 2026. Gestión de código fuente.

[6] Google. Material Design 3. https://m3.material.io/, 2026. Sistema de diseño utilizado en la aplicación.

[7] Google Developers. Corrutinas de Kotlin en Android. https://developer.android.com/kotlin/coroutines?hl=es-419, 2026. Gestión de tareas asíncronas.

[8] Google Developers. Introducción a Android Studio. https://developer.android.com/studio/intro?hl=es-419, 2026. Manual del entorno de desarrollo.

[9] Google Developers. Jetpack Compose: Interfaces nativas modernas. https://developer.android.com/compose, 2026. Documentación del kit de UI.

[10] Google Developers. Media3 ExoPlayer: Reproducción multimedia. https://developer.android.com/media/media3/exoplayer?hl=es-419, 2026. Librería para la visualización de vídeos.

[11] JGraph Ltd. Draw.io: Herramienta de creación de diagramas. https://www.drawio.com/, 2026. Software utilizado para el modelado de datos.

[12] Mayo Clinic. Enfermedad de Parkinson: Síntomas y causas. https://www.mayoclinic.org/es/diseases-conditions/parkinsons-disease/symptoms-causes/syc-20376055, 2026. Referencia médica sobre la patología.

[13] Parkinson's Foundation. Bradicinesia: Síntomas motores del Parkinson. https://www.parkinson.org/espanol/entendiendo-parkinson/sintomas-motores/bradicinesia, 2026. Definición técnica de la lentitud de movimiento.

[14] Python Software Foundation. Python Programming Language. https://www.python.org/, 2026. Sitio oficial del lenguaje del Backend.

[15] Stack Overflow. Comunidad de desarrollo y resolución de problemas en Kotlin. https://stackoverflow.com/questions/tagged/kotlin?tab=Newest, 2026. Recurso de consulta para desarrolladores.

[16] The PostgreSQL Global Development Group. PostgreSQL: La base de datos relacional más avanzada. https://www.postgresql.org/, 2026. Motor de persistencia de datos.

[17] UNIR Revista. ¿Qué es un framework en programación? https://unirfp.unir.net/revista/ingenieria-y-tecnologia/framework/, 2026. Conceptos de ingeniería de software.

[18] Wikipedia. mHealth (Salud móvil). https://es.wikipedia.org/wiki/MHealth_(salud_m%C3%B3vil), 2026. Conceptos base de salud mediante dispositivos móviles.